



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 04

Module Objective: In previous weeks, your Drow Ranger agent used A* Search to find a physical path to the goal. However, it blindly assumed every non-wall tile was safe. Today, we implement a **Knowledge Base (KB)** and a **Forward Chaining Inference Engine**. Your agent must now mathematically prove a cell is logically *Feasible* before taking a step, using real-time rules about map vision and enemy threats.

Part 1: The Logic Engine Architecture

Step 1.1: Building the Knowledge Base (KB)

Instead of hardcoding hundreds of nested `if-else` statements in our agent's movement logic, we will build a declarative Knowledge Base. You need to design a Python class that stores **Facts** (current percepts) and **Rules** (Horn Clauses).

1. Create a new file named `logic_engine.py` .
2. Implement a `KnowledgeBase` class based on the following structural requirements:

```
CLASS KnowledgeBase:
    // Attributes
    DECLARE facts AS Set           // To store unique string fact
    DECLARE rules AS List         // To store rules as Tuples: (

    // Methods
    FUNCTION tell_fact(fact_string):
        ADD fact_string TO facts

    FUNCTION tell_rule(premise_list, conclusion_string):
        APPEND Tuple(premise_list, conclusion_string) TO rules
```

```
FUNCTION clear_facts():  
    EMPTY the facts Set
```

Part 2: Implementing Forward Chaining

Step 2.1: The Inference Algorithm

The agent needs an Inference Engine to deduce new information from its raw sensors. Translate the following Data-Driven **Forward Chaining** algorithm into a Python method inside your `KnowledgeBase` class.

1. Write the `forward_chain()` method.
2. Ensure you use a `while` loop that continues to iterate over the rules until a full pass results in absolutely no new facts being deduced.

```
FUNCTION forward_chain():  
    DECLARE new_facts_added = TRUE  
  
    WHILE new_facts_added == TRUE:  
        new_facts_added = FALSE  
  
        FOR EACH (premises, conclusion) IN rules:  
            IF conclusion IS NOT IN facts:  
  
                // Modus Ponens Check  
                IF ALL premises ARE IN facts:  
                    ADD conclusion TO facts  
                    new_facts_added = TRUE
```

Part 3: Hooking Logic into the Grid Game

Step 3.1: Defining the Game Constraints

Now we translate the game's safety constraints into our logic engine using Propositional Logic.

1. In your main `agent.py`, instantiate the KB in the `__init__` method: `self.kb = KnowledgeBase()`.
2. Use your `tell_rule` method to define the following safety rules:
Rule 1: `TargetVisible  $\wedge$  HasDust  $\Rightarrow$  SafeToEngage`
Rule 2: `SafeToEngage  $\wedge$  BloodseekerMissing  $\Rightarrow$  Retreat`

Step 3.2: Validating Feasibility in A*

Modify your existing A* pathfinding algorithm to consult the Knowledge Base before expanding a node.

1. Locate the neighbor-generation loop in your A* code (where you normally check for physical walls).
2. Before adding a neighbor to the `open_list`, clear the KB facts.
3. Feed the current percepts for that specific tile into the KB using `tell_fact()`.
4. Run `self.kb.forward_chain()`.
5. If `'Retreat'` is deduced and exists in `self.kb.facts`, mark the tile as **Infeasible**. Skip it (do not add it to the open list), even if it is physically reachable.

Part 4: Self-Evaluation Test Cases

4.1 Automated Validation

Create a file named `test_logic.py`. Copy and run the following Python assert statements to verify your Forward Chaining engine works correctly mathematically before you attempt to integrate it into the visual grid.

```
from logic_engine import KnowledgeBase

def test_forward_chaining():
    kb = KnowledgeBase()

    # Add Domain Rules
    kb.tell_rule(['TargetVisible', 'HasDust'], 'SafeToEngage')
    kb.tell_rule(['SafeToEngage', 'BloodseekerMissing'], 'Retreat')

    # Test Case 1: Safe Engagement
    kb.clear_facts()
    kb.tell_fact('TargetVisible')
    kb.tell_fact('HasDust')
    kb.forward_chain()
    assert 'SafeToEngage' in kb.facts, "Test 1 Failed: Should deduce Sa
    assert 'Retreat' not in kb.facts, "Test 1 Failed: Should NOT deduce

    # Test Case 2: Unsafe Engagement (Bloodseeker Missing)
    kb.clear_facts()
    kb.tell_fact('TargetVisible')
    kb.tell_fact('HasDust')
    kb.tell_fact('BloodseekerMissing')
    kb.forward_chain()
    assert 'Retreat' in kb.facts, "Test 2 Failed: Should deduce Retreat

    print("✅ All Logic Engine Test Cases Passed!")

if __name__ == "__main__":
    test_forward_chaining()
```

Part 5: Theory-to-Implementation Mapping

Based on the code you just wrote and the lecture on Logical Reasoning, complete the following analytical questions to explain *how* your code implements the theory.

1. Reachability vs. Feasibility (Apply)

In Step 3.2, you modified the A* algorithm's neighbor-generation loop. How does your

new code differentiate between checking if a node is "Reachable" (like a wall collision) versus checking if it is "Feasible"? Explain how the Knowledge Base enforces feasibility.

The key distinction is:

- **Reachable** asks: *Can the agent physically move to this neighboring node?*
The modified A* neighbor-generation loop checks basic movement constraints, such as whether the node is **inside the map and not a wall/obstacle**. If it collides with a wall, it is not reachable and is skipped immediately.
- **Feasible** asks: *Even if the agent can physically move there, is that move allowed according to the problem's rules/Knowledge Base?*
A node can therefore be reachable but infeasible—for example, if entering it violates a known constraint or precondition.

The **Knowledge Base (KB) enforces feasibility** by providing the rules/constraints that the neighbor-generation code must satisfy. After the basic reachability check succeeds, the code consults the KB (directly or through the feasibility-checking function). Only neighbors that satisfy those KB rules are added to the A* open set.

2. Declarative vs. Procedural Paradigms (Analyze)

Why did we build a `KnowledgeBase` class with a `forward_chain()` loop instead of simply writing `if 'TargetVisible' in percepts` and `'HasDust' in percepts`: directly inside the agent's movement code? What happens to the agent's code complexity if we add 50 new game rules?

Building a `KnowledgeBase` shifts the design from a procedural paradigm, where rules are hardcoded into control flow, to a declarative paradigm, where domain knowledge is separated from the execution engine.

Decoupling Logic and Execution: Hardcoding conditions directly inside the movement code mixes domain rules with the agent's execution logic. A declarative `KnowledgeBase` stores facts and rules as data structures, allowing the inference engine (`forward_chain()`) to evaluate those rules without changing the agent's core search or movement logic.

Managing Code Complexity: If 50 new game rules were added procedurally, the agent's movement code could become filled with many `if-else` statements, boolean checks, and dependencies on execution order, making it harder to maintain and debug. With the declarative approach, the new rules can be registered using `tell_rule()` , while the existing `forward_chain()`

mechanism processes them automatically. This keeps the agent's core code relatively stable as the number of domain rules increases.

Therefore, the main benefit is **separation of concerns and maintainability**: the Knowledge Base contains *what the rules are*, while the inference engine determines *how those rules are applied*.

3. Modus Ponens & Horn Clauses (Understand)

The rule `['TargetVisible', 'HasDust'] ⇒ 'SafeToEngage'` is formally known as a Horn Clause. Briefly explain how your Python implementation of the `forward_chain()` `while` loop acts as a mathematical execution of the *Modus Ponens* inference rule.

Identification of Premise Matching: The `forward_chain()` method iterates through the rule base and evaluates the condition `all(p in self.facts for p in premises)`, verifying that every antecedent, such as `P` and `Q`, of a Horn clause is present in the current set of facts.

Application of Consequent Inference: If all premises are satisfied, the algorithm applies **Modus Ponens** by adding the rule's conclusion `R` (e.g., `SafeToEngage`) to the set of known facts.

Iterative Closure: The outer `while` loop, controlled by the `new_facts_added` flag, ensures that newly inferred facts can trigger subsequent rules. The process continues until no new facts can be derived, reaching a fixed point known as the **logical closure** of the knowledge base.

Thus, the `forward_chain()` loop is a computational implementation of Modus Ponens: whenever the premises of a rule are known to be true, its conclusion is inferred and added to the knowledge base.

Finalize and Lock Lab 05 Implementation



FACULTY OF COMPUTING

